

GCC1734 – Inteligência Artificial

Notas de Aula

Busca sem Informação

Prof. Eduardo Bezerra

Sumário

1	Estratégias de Busca sem Informação	3
2	Busca em Largura (BFS)	4
2.1	Ideia e Implementação	4
2.2	Exemplo de Execução	5
2.3	Propriedades	6
3	Busca com Custo Uniforme (UCS)	8
3.1	Ideia e Implementação	8
3.2	Exemplo de Execução	8
3.3	Propriedades	10
4	Busca em Profundidade (DFS)	11
4.1	Ideia e Implementação	11
4.2	Exemplo de Execução	11
4.3	Propriedades	12
5	Busca em Profundidade Limitada (DLS)	14
5.1	Ideia e Implementação	14
5.2	Exemplo de Execução	15
5.3	Propriedades	16
6	Busca em Profundidade Iterativa (IDS)	17
6.1	Ideia e Implementação	17
6.2	Exemplo de Execução	17
6.3	A Questão do Retrabalho	18
6.4	Propriedades	18
7	Comparação entre Estratégias	19
8	Considerações Finais	21

9	Atividades Intra-Classe	22
9.1	Exercício 1: BFS Manual	22
9.2	Exercício 2: DFS Manual	22
9.3	Exercício 3: UCS Manual	22
9.4	Exercício 4: DLS e IDS Manuais	22
9.5	Exercício 5: Escolha do Algoritmo	22

Objetivos

Ao final da leitura, você deve ser capaz de:

- Distinguir estratégias de busca sem informação das demais estratégias.
- Descrever o funcionamento dos algoritmos BFS, UCS, DFS, DLS e IDS.
- Analisar cada algoritmo segundo as propriedades de completude, otimalidade e complexidade de tempo e espaço.
- Implementar cada estratégia a partir do esquema genérico de busca, variando apenas a estrutura de dados da fronteira.
- Explicar a diferença entre busca em árvore e busca em grafo, e justificar o uso do conjunto de estados explorados.

1 Estratégias de Busca sem Informação

Nas aulas anteriores, definimos formalmente um problema de busca como uma 6-tupla da forma $\langle S, \text{ACTIONS}, T, c, s_0, \text{GoalTest} \rangle$ e apresentamos o esquema genérico de resolução. Nesse esquema, a única diferença entre algoritmos é a regra de seleção da estrutura de dados utilizada para representar a *fronteira*. Chegou o momento de instanciar esse esquema em algoritmos concretos.

As *estratégias de busca sem informação*, também chamadas de *busca cega* (*blind search*), são aquelas que utilizam *apenas* a informação presente na formulação do problema para escolher qual nó da árvore de busca expandir a seguir. Elas não possuem nenhum conhecimento adicional sobre o domínio, tal como estimativas de distância ao objetivo ou características particulares dos estados. Para uma estratégia cega, todos os estados não-objetivo são equivalentes entre si: não há como distinguir um estado “promissor” de um “sem saída” sem explorá-los.

A consequência prática é que a única variável de projeto dessas estratégias é a *política de seleção da fronteira*: qual nó dentre os gerados será expandido primeiro. Cada escolha produz um algoritmo com características distintas de desempenho, uso de memória, completude e otimalidade.

As principais estratégias de busca sem informação são listadas a seguir. Cada uma delas será apresentada nas seções seguintes, com análise de suas propriedades formais.

- Busca em Largura (BFS),
- Busca com Custo Uniforme (UCS),
- Busca em Profundidade (DFS),
- Busca em Profundidade Limitada (DLS),
- Busca em Profundidade Iterativa (IDS).

Avaliando algoritmos de busca

Quatro critérios são usados para avaliar algoritmos de busca:

- **Compleitude:** o algoritmo sempre encontra uma solução se ela existir?
- **Otimalidade:** se encontrar uma solução, ela tem custo mínimo?
- **Complexidade de tempo:** quantos nós são gerados ou expandidos ao longo da busca?
- **Complexidade de espaço:** quantos nós precisam estar em memória simultaneamente?

As complexidades são expressas em termos de b (fator de ramificação), d (profundidade da solução mais rasa) e m (profundidade máxima da árvore de busca).

2 Busca em Largura (BFS)

2.1 Ideia e Implementação

A *Busca em Largura* (*Breadth-First Search*, BFS) é a instância mais natural do esquema genérico de busca: expande sempre o nó mais próximo da raiz na árvore de busca. Antes de explorar qualquer nó na profundidade $d + 1$, todos os nós na profundidade d já foram expandidos. A BFS percorre o espaço de estados “em camadas horizontais”, avançando de nível em nível.

A implementação é direta: a fronteira é uma fila FIFO (*first-in, first-out*). Novos nós são inseridos no final da fila, e o nó selecionado para expansão é sempre o que está na frente, ou seja, o que foi gerado há mais tempo. Como nós mais antigos estão em camadas mais rasas, isso garante que a busca processe os níveis em ordem crescente de profundidade.

Uma observação importante sobre a implementação: na BFS, o *teste de objetivo pode ser aplicado quando o nó é gerado* (em vez de quando é selecionado para expansão). Isso é seguro porque a BFS explora a árvore em ordem crescente de profundidade. Assim, quando um nó objetivo é gerado pela primeira vez, ele necessariamente está na menor profundidade possível.

Essa implementação pressupõe que estamos interessados na solução de *menor profundidade*, o que coincide com a solução de menor custo quando todas as ações têm custo uniforme. Em problemas com custos variáveis, essa propriedade deixa de valer, e o teste de objetivo deve ser aplicado apenas quando o nó é selecionado para expansão, como faz a Busca de Custo Uniforme (descrita mais adiante).

Busca em árvore vs. busca em grafo nos exemplos

Nos exemplos desta seção utilizaremos duas variantes do esquema de busca.

BFS e UCS serão apresentados na forma de *busca em grafo*: estados repetidos não são reinseridos na fronteira. Isso evita expansões redundantes e torna a execução mais fácil de acompanhar.

Já os exemplos de **DFS, DLS e IDS** serão apresentados no estilo de *busca em árvore* (isto é, sem um conjunto global de estados explorados). Ainda assim, para evitar ciclos triviais na execução ilustrativa, assumiremos uma verificação mínima: **o algoritmo não gera um filho que já esteja no caminho atual** da raiz até o nó sendo expandido (*path-check*). Isso preserva a intuição de *backtracking* sem introduzir uma estrutura extra como o *explored set*.

Essa diferença afeta apenas a execução ilustrativa; as propriedades assintóticas discutidas para cada algoritmo permanecem válidas.

2.2 Exemplo de Execução

Para acompanhar a execução da BFS passo a passo, usaremos o grafo da Figura 1, com estado inicial S e estado objetivo G . Os filhos de cada nó são sempre listados em ordem alfabética. Este mesmo grafo será reutilizado nas seções seguintes para comparar o comportamento das diferentes estratégias de busca.

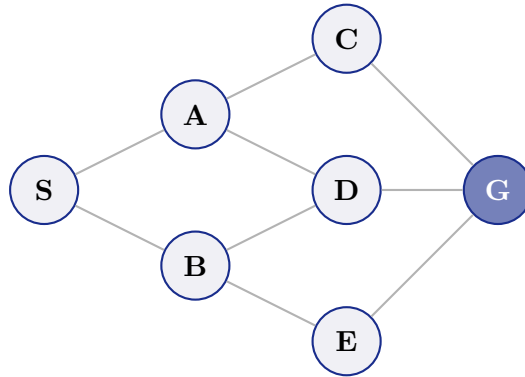


Figura 1: Grafo usado no exemplo de BFS. Estado inicial: S ; estado objetivo: G (azul-escuro). Os filhos de cada nó são considerados em ordem alfabética.

A Tabela 1 mostra a evolução da fronteira a cada iteração. Lembre-se: a fronteira é uma fila FIFO; o nó retirado é sempre o da *frente* da fila, e os filhos gerados são inseridos no *final*.

Perceba como a fila FIFO *força* a expansão em camadas: S (profundidade 0) é expandido antes de A e B (profundidade 1), que por sua vez são expandidos antes de C , D e E (profundidade 2). Isso garante que, quando G é gerado pela primeira vez, estamos exatamente na profundidade mínima em que ele pode ser alcançado.

A Figura 2 mostra o estado da busca ao final da iteração 4, no instante em que G é gerado como filho de C e a busca é encerrada: S , A e B já haviam sido expandidos; D e E permanecem na fronteira, mas não chegam a ser expandidos.

O caminho retornado é $S \rightarrow A \rightarrow C \rightarrow G$, com 3 arestas. Note que existem outros caminhos para G com o mesmo comprimento ($S \rightarrow A \rightarrow D \rightarrow G$ e $S \rightarrow B \rightarrow D \rightarrow G$), e ainda um com 3 arestas passando por E . A BFS encontra *um* dos caminhos de menor

Iter.	Nó removido da fronteira	Filhos gerados	Fronteira após expansão
0	–	–	$[S]$
1	S	A, B	$[A, B]$
2	A	C, D	$[B, C, D]$
3	B	E	$[C, D, E]$
4	C	$G \dagger$	<i>busca encerrada</i>

$\dagger$ G é gerado na iteração 4; o teste de objetivo é aplicado *na geração* e a busca encerra imediatamente.

Tabela 1: Execução passo a passo da BFS no grafo da Figura 1. A fila FIFO é mostrada da esquerda (frente) para a direita (fim). Como o teste de objetivo é aplicado na geração, a busca é encerrada assim que G é produzido. Solução encontrada: $S \rightarrow A \rightarrow C \rightarrow G$.

profundidade; qual deles depende da ordem de expansão dos filhos. Qualquer um deles é igualmente válido como solução de profundidade mínima.

2.3 Propriedades

Compleitude

A BFS é completa se o fator de ramificação b for finito. Se existe uma solução na profundidade d , a BFS irá inevitavelmente gerar todos os nós das profundidades $0, 1, \dots, d$, encontrando a solução.

Otimalidade

A BFS é ótima quando o custo de cada ação é idêntico, isto é, quando $c(s, a) = k$ para alguma constante $k > 0$. Nesse caso, o custo total é proporcional à profundidade, e a BFS (ao expandir em ordem crescente de profundidade) garante encontrar o nó-objetivo de menor custo primeiro. Se os custos forem heterogêneos, a BFS não é mais ótima (o que motiva a UCS, apresentada a seguir).

Complexidade

Seja d a profundidade da solução mais rasa e b o fator de ramificação. No pior caso, a BFS expande todos os nós até a profundidade d , gerando $1 + b + b^2 + \dots + b^d = O(b^d)$ nós. A complexidade de tempo é $O(b^d)$.

A complexidade de espaço é dominada pelo tamanho da fronteira. No momento em que a BFS processa o nível d , a fronteira contém todos os nós desse nível, totalizando $O(b^d)$ nós simultâneos em memória.

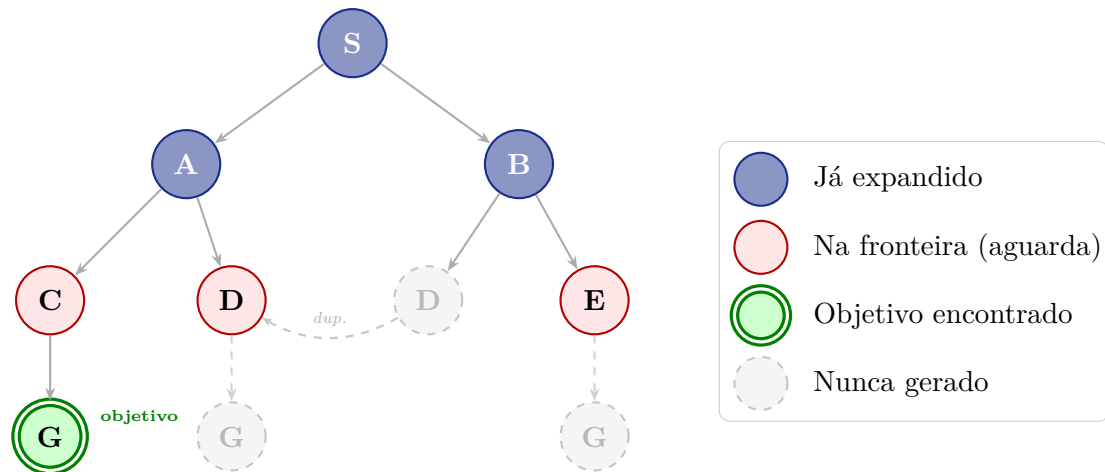


Figura 2: Árvore de busca gerada pela BFS ao final da iteração 4. Até esse ponto, S , A e B já haviam sido expandidos, e C acaba de gerar o nó-objetivo G . O nó D filho de B (tracejado cinza) nunca chega a ser gerado: a busca em grafo detecta que D já está na fronteira antes de inseri-lo novamente. A seta *dup.* indica essa identificação. Os nós G filhos de D e de E (tracejado, arestas tracejadas) *nunca chegam a ser gerados*: o teste de objetivo é aplicado na geração, e ao gerar G como filho de C a busca encerra imediatamente. Isso ilustra a diferença essencial entre a *árvore de busca* (onde um mesmo estado pode aparecer em múltiplos nós) e o *grafo de estados* subjacente. Solução encontrada: $S \rightarrow A \rightarrow C \rightarrow G$.

O gargalo da BFS é a memória

A tabela a seguir ilustra os requisitos de tempo e espaço da BFS com $b = 10$, supondo 10^6 nós por segundo e 1 KB por nó.

Profundidade d	Nós	Tempo	Memória
2	$\approx 10^2$	< 1 ms	≈ 100 KB
4	$\approx 10^4$	10 ms	≈ 10 MB
6	$\approx 10^6$	1 s	≈ 1 GB
8	$\approx 10^8$	100 s ($\approx 1,7$ min)	≈ 100 GB
10	$\approx 10^{10}$	10^4 s ($\approx 2,8$ h)	≈ 10 TB
12	$\approx 10^{12}$	10^6 s ($\approx 11,6$ dias)	≈ 1 PB

A lição principal é que os requisitos de memória são o verdadeiro gargalo da BFS no pior caso: a fronteira pode crescer como b^d e esgotar o espaço físico disponível muito antes do tempo de processamento.

Na prática, porém, muitos domínios (como labirintos em grade) têm *fator* b pequeno e um número total de estados $|S|$ limitado; com busca em grafo, cada estado é expandido no máximo uma vez, e a BFS pode funcionar bem mesmo quando a solução tem mais de alguns poucos passos.

Quando a BFS funciona bem na prática

Embora a complexidade teórica da BFS seja $O(b^d)$, em muitos problemas práticos, o número total de estados distintos $|S|$ é limitado. Quando a busca é implementada como busca em grafo, cada estado é expandido no máximo uma vez, e o custo passa a ser $O(|S| + |E|)$.

Esse é o caso típico de problemas em grade (labirintos), onde diferentes caminhos frequentemente levam ao mesmo estado.

Pergunta 1

A BFS garante encontrar a solução de **menor profundidade**. Mas profundidade mínima não é o mesmo que custo mínimo. Dê um exemplo simples (com 3–4 nós e custos variados) em que a solução de menor profundidade **não** é a de menor custo. Explique por que a BFS falha em encontrar a solução ótima nesse cenário.

Pergunta 2

Por que a **complexidade de espaço** é o maior problema da BFS na prática, e não a complexidade de tempo? Use os dados da tabela acima para justificar sua resposta em duas ou três frases.

3 Busca com Custo Uniforme (UCS)

3.1 Ideia e Implementação

A *Busca com Custo Uniforme* (*Uniform-Cost Search*, UCS) generaliza a BFS para problemas em que as ações têm *custos diferentes*. Em vez de expandir o nó mais raso da fronteira, a UCS expande sempre o nó com menor custo acumulado $g(n)$ desde o estado inicial. Assim, a busca deixa de priorizar profundidade e passa a priorizar o *custo total do caminho já percorrido*.

A implementação usa uma *fila de prioridade* ordenada por $g(n)$. Sempre que um novo nó é gerado, ele é inserido na fronteira com sua chave de prioridade igual ao custo acumulado do caminho que o alcança. O próximo nó a ser expandido é sempre o de menor g .

Diferentemente da BFS, na UCS o *teste de objetivo deve ser aplicado quando o nó é selecionado para expansão*, e não no momento da geração. A razão é simples: o primeiro nó objetivo *gerado* pode não corresponder ao caminho de menor custo. Apenas quando um nó objetivo chega ao topo da fila de prioridade é que podemos garantir que nenhum outro caminho mais barato até ele permanece por explorar.

Essa estratégia torna a UCS *ótima para custos não-negativos* e explica por que ela é a escolha correta quando a solução desejada é a de menor custo, e não necessariamente a de menor profundidade.

3.2 Exemplo de Execução

Usaremos o mesmo grafo da Figura 1, repetido aqui como Figura 3 por conveniência, agora com custos heterogêneos atribuídos às arestas. O estado inicial é S e o estado objetivo é G .

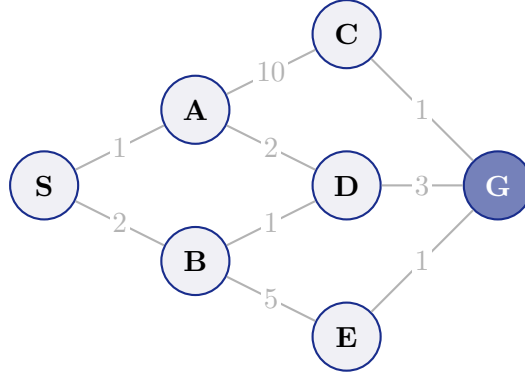


Figura 3: Grafo de espaço de estados usado no exemplo de UCS (mesma topologia da Figura 1), agora com custos heterogêneos. Estado inicial: S ; estado objetivo: G (azul-escuro). Os filhos de cada nó são considerados em ordem alfabética.

A Tabela 2 mostra a evolução da fronteira a cada iteração. Lembre-se: a fronteira é uma fila de prioridade ordenada por $g(n)$; o nó retirado é sempre o de *menor custo acumulado*.

Iter.	Nó removido da fronteira	Filhos gerados	Fila de prioridade após expansão
0	–	–	$[(S, 0)]$
1	S ($g=0$)	A ($g=1$), B ($g=2$)	$[(A, 1), (B, 2)]$
2	A ($g=1$)	C ($g=11$), D ($g=3$)	$[(B, 2), (D, 3), (C, 11)]$
3	B ($g=2$)	D ($g=3$) †, E ($g=7$)	$[(D, 3), (E, 7), (C, 11)]$
4	D ($g=3$)	G ($g=6$)	$[(G, 6), (E, 7), (C, 11)]$
5	G ($g=6$)	<i>objetivo!</i>	–

† D já está na fila com $g=3$ (via A). O caminho via B também chega com $g = 2 + 1 = 3$; como os custos são iguais, não há substituição.

Tabela 2: Execução passo a passo da UCS no grafo da Figura 3. A fila de prioridade é mostrada em ordem crescente de g (menor custo à esquerda). Solução encontrada: $S \rightarrow A \rightarrow D \rightarrow G$, custo total 6.

O resultado contrasta diretamente com o da BFS no mesmo grafo. A BFS expande A antes de B e gera C como primeiro filho de A ; como o teste de objetivo é aplicado na *geração*, o primeiro nó-objetivo encontrado pela BFS seria G via C (profundidade 3, custo $1 + 10 + 1 = 12$). A UCS ignora por completo a ordem de geração e expande sempre o nó de menor g : ao chegar à iteração 4, o nó D (custo acumulado 3) está na frente da fila, e seu filho G é alcançado com $g = 6$.

Dois momentos merecem atenção especial.

O primeiro ocorre na **iteração 3**: B é expandido e gera D com $g = 2 + 1 = 3$. Como D já está na fila com $g = 3$ (chegou via A na iteração anterior), os dois caminhos têm custo idêntico; nenhuma substituição ocorre, e a cópia via A permanece na fila. Esse empate é incomum; em geral, um dos caminhos seria mais barato e o mais caro seria descartado.

O segundo momento decisivo é a **iteração 4**: D é retirado da fila e gera G com $g = 3 + 3 = 6$. Na **iteração 5**, G é o nó de menor custo na fila, é selecionado para

expansão, o teste de objetivo é aplicado e a busca encerra.

O caminho encontrado, $S \rightarrow A \rightarrow D \rightarrow G$ (custo 6), não é o de menor profundidade: existem caminhos de profundidade 3 com custos muito maiores, como $S \rightarrow A \rightarrow C \rightarrow G$ (custo 12). A UCS escolhe corretamente o caminho de menor custo; em caso de empate no custo acumulado, o caminho retornado depende da ordem de inserção na fila — comportamento distinto da BFS, que ignora custos por completo.

A Figura 4 mostra a árvore de busca construída pela UCS ao final da execução.

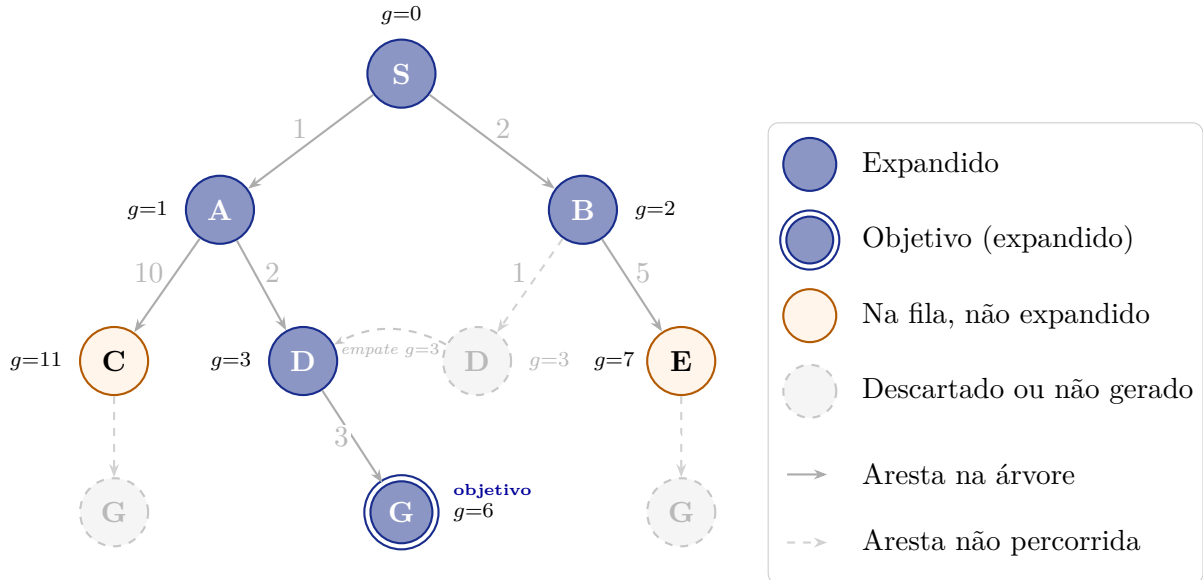


Figura 4: Árvore de busca construída pela UCS ao final da execução. O nó D filho de B (tracejado cinza) foi gerado mas descartado por empate de custo: ambos os caminhos chegam com $g=3$, e a cópia via A já estava na fila. Os nós G filhos de C e de E *nunca são gerados*: na UCS, o teste de objetivo é aplicado na *expansão* (não na geração), e G é expandido na iteração 5 antes que C ou E sejam retirados da fila. O caminho ótimo é $S \rightarrow A \rightarrow D \rightarrow G$ (custo 6): embora D também seja alcançável via B com o mesmo custo acumulado $g=3$, a cópia via A chegou primeiro à fila e é a que permanece na árvore. Esse caminho contrasta com $S \rightarrow A \rightarrow C \rightarrow G$ (custo 12), que a BFS teria retornado por ser o de menor profundidade.

3.3 Propriedades

Completeness

A UCS é **completa** se o custo de cada ação for maior ou igual a alguma constante positiva $\varepsilon > 0$. Essa condição evita que a busca fique presa em ciclos de custo zero ou que a sequência de custos convirja para um valor finito com infinitas ações.

Optimality

A UCS é **ótima** em geral. A intuição é: como os nós são sempre expandidos em ordem crescente de $g(n)$, quando um nó objetivo é selecionado para expansão, não existe outro caminho até esse estado com custo menor (pois esse caminho teria sido expandido antes, por ter menor g). Isso pressupõe custos não-negativos.

Complexidade

A complexidade da UCS é expressa em termos do custo ótimo C^* e do custo mínimo de uma ação ε . A razão $\lfloor C^*/\varepsilon \rfloor$ aproxima a “profundidade efetiva” da solução ótima. Tanto a complexidade de tempo quanto a de espaço são $O(b^{\lfloor C^*/\varepsilon \rfloor})$. Quando os custos são uniformes, isso se reduz a $O(b^d)$, como na BFS. Em geral, porém, a UCS pode ser muito mais custosa que a BFS se houver muitas ações de custo baixo.

UCS vs. BFS

A BFS encontra a solução de menor *profundidade*; a UCS encontra a de menor *custo total*. Quando os custos são todos iguais, as duas propriedades coincidem. Mas quando os custos variam, a BFS pode retornar um caminho raso e caro, enquanto a UCS busca o caminho mais barato independentemente de quão fundo ele esteja na árvore.

Pergunta 3

Construa um exemplo simples (3–5 nós) em que a UCS expande **mais** nós do que a BFS antes de encontrar a solução. Explique por que isso ocorre: qual característica dos custos das ações leva a esse comportamento?

4 Busca em Profundidade (DFS)

4.1 Ideia e Implementação

A Busca em Profundidade (*Depth-First Search*, DFS) opera pela lógica oposta à BFS: em vez de expandir o nó mais raso da fronteira, expande sempre o *nó mais recentemente gerado*, ou seja, o mais profundo na árvore de busca no momento. A DFS mergulha o mais fundo possível em cada ramo antes de retroceder (*backtrack*) e explorar alternativas.

A implementação usa uma *pilha LIFO* (*last-in, first-out*) como fronteira. Novos nós filhos são empilhados, e o nó no topo (i.e., o gerado mais recentemente) é sempre o próximo a ser expandido. Quando um nó não tem filhos não visitados (folha), ele é descartado e o próximo nó no topo da pilha é selecionado: isso é o *backtrack*.

4.2 Exemplo de Execução

Usaremos o mesmo grafo da Figura 1, já apresentado na seção anterior. O estado inicial é S e o estado objetivo é G . Os filhos de cada nó são considerados em ordem alfabética. Na pilha LIFO, o filho que deve ser expandido primeiro precisa ser inserido por último.

A Tabela 3 mostra a evolução da pilha LIFO a cada iteração. O nó no topo da pilha (à esquerda na notação adotada) é sempre o próximo a ser expandido.

Nota. Neste exemplo aplicaremos o teste de objetivo *na geração* apenas para simplificar a leitura da execução. Em implementações gerais (especialmente em grafos com ciclos), é comum aplicar o teste de objetivo quando o nó é removido da fronteira, além de usar verificações para evitar repetições.

Neste grafo a DFS encontra G rapidamente, sem precisar fazer *backtrack*. Para ver o *backtrack* em ação, basta imaginar que G não fosse filho de C : a DFS chegaria à folha C , a descartaria da pilha e continuaria com D (próximo no topo), retrocedendo um nível antes de mergulhar novamente.

Iter.	Nó removido da fronteira	Filhos gerados	Pilha após expansão (topo à esquerda)
0	–	–	$[S]$
1	S	A, B	$[A, B]$
2	A	C, D	$[C, D, B]$
3	C	$G \dagger$	<i>busca encerrada</i>

$\dagger$ G é gerado na iteração 3; o teste de objetivo é aplicado na geração e a busca encerra imediatamente.

Tabela 3: Execução passo a passo da DFS no grafo da Figura 1. Solução encontrada: $S \rightarrow A \rightarrow C \rightarrow G$. A pilha LIFO garante que o filho mais recentemente gerado (C , topo) é expandido antes de D e B , que aguardam mais abaixo. Como o teste de objetivo é aplicado na geração neste exemplo, a busca é encerrada assim que G é produzido.

A Figura 5 mostra o instantâneo da árvore de busca no início da iteração 3, quando C está prestes a ser expandido e D e B aguardam na pilha.

4.3 Propriedades

Compleitude

A DFS não é completa na variante de *busca em árvore*. Se o espaço de estados contiver ciclos, o algoritmo pode revisitar os mesmos estados indefinidamente, entrando em um loop infinito. Mesmo em grafos sem ciclos, a presença de ramos de profundidade infinita pode impedir que a busca alcance outras regiões do espaço.

Quando implementada como *busca em grafo*, mantendo um conjunto de estados já explorados, a DFS termina em espaços de estados finitos. Ainda assim, ela pode explorar ramos muito profundos antes de alcançar uma solução rasa, o que a torna pouco previsível em termos de desempenho.

Otimalidade

A DFS não é ótima. Ela encontra a solução “mais à esquerda” na árvore de busca (aquela gerada primeiro pela ordem de expansão dos filhos), independentemente de sua profundidade ou custo. Essa solução pode ser muito mais cara do que outras soluções existentes.

Complexidade

A complexidade de tempo da DFS é $O(b^m)$, onde m é a profundidade máxima da árvore, o que pode ser muito pior que a BFS se $m \gg d$.

A complexidade de espaço é $O(b \cdot m)$, ou seja, espaço *linear* em m . Essa é a grande vantagem da DFS: em qualquer instante, apenas o caminho atual da raiz até o nó sendo expandido, e os irmãos não expandidos ao longo desse caminho, precisam estar em memória. Isso é exatamente o que a Figura 5 ilustra: somente S , A , C (caminho atual) e D , B (irmãos pendentes) ocupam memória; os demais nós ainda não existem.

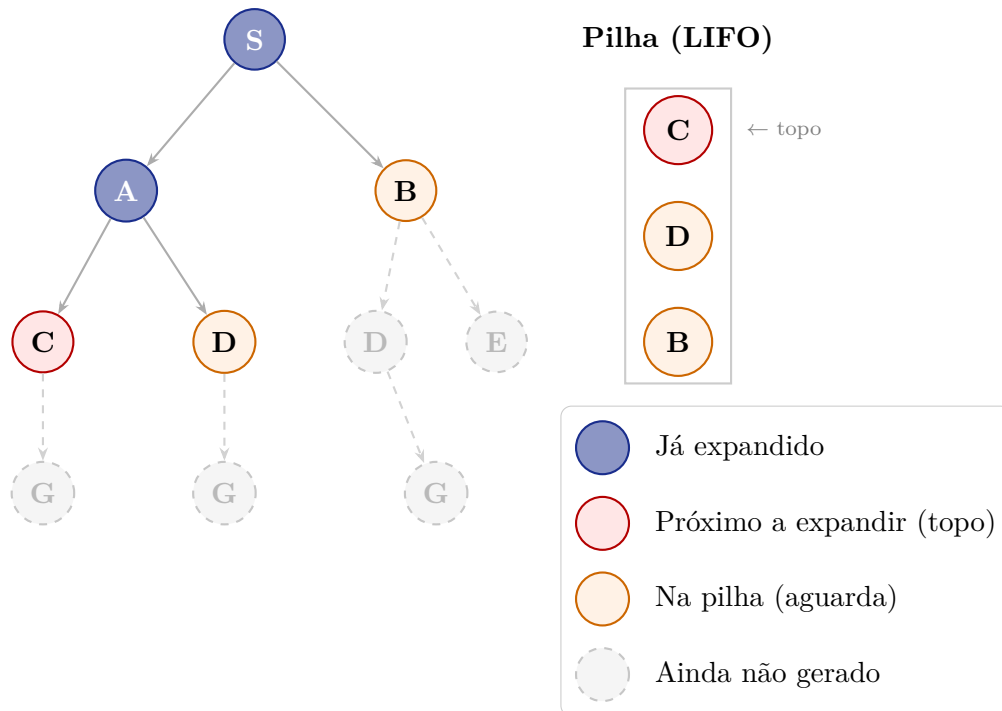


Figura 5: Instantâneo da árvore de busca gerada pela DFS no início da iteração 3. S e A já foram expandidos (azul); C está no topo da pilha e será expandido a seguir (vermelho); D e B aguardam na pilha (laranja). Os filhos de B (D e E) e todos os nós da camada 3 (*tracejado cinza*) ainda não foram gerados, pois B não foi expandido. O diagrama lateral mostra o conteúdo da pilha LIFO nesse momento. Note que apenas o caminho atual $S \rightarrow A$ e os nós em espera (D , B) precisam estar em memória, ilustrando o custo $O(b \cdot m)$ da DFS.

A grande vantagem da DFS: uso linear de memória

Com $b = 10$ e $m = 12$, a BFS exige cerca de 1 petabyte de memória ($O(b^m)$), enquanto a DFS requer apenas $10 \times 12 = 120$ nós no pior caso ($O(b \cdot m)$). Essa diferença de muitas ordens de grandeza torna a DFS viável em muitos problemas práticos onde a BFS seria impossível.

Pergunta 4

A DFS pode entrar em **loop infinito** em grafos com ciclos. Descreva um cenário concreto (com dois ou três estados conectados por ciclo) em que isso ocorre. Em seguida, explique o que é necessário acrescentar à implementação para evitar esse comportamento.

Pergunta 5

A DFS usa espaço $O(b \cdot m)$, enquanto a BFS usa $O(b^d)$. Para $b = 10$ e $d = m = 8$, calcule o número de nós em memória para cada estratégia. O que essa diferença revela sobre quando é preferível usar DFS?

5 Busca em Profundidade Limitada (DLS)

5.1 Ideia e Implementação

A DFS tem como principal limitação a possibilidade de mergulhar indefinidamente em ramos sem solução. Uma forma simples de contornar esse problema é impor um *limite de profundidade* L : o algoritmo nunca expande nós além da profundidade L . Essa variante é chamada de *Busca em Profundidade Limitada* (*Depth-Limited Search*, DLS).

A DLS funciona exatamente como a DFS, exceto que nós na profundidade L são tratados como folhas, independentemente de terem filhos. O algoritmo pode retornar três resultados:

- **Solução**: um plano satisfazendo o *GoalTest* foi encontrado dentro do limite.
- **Falha** (*failure*): todos os ramos dentro do limite foram explorados sem encontrar o objetivo; o objetivo *não existe* dentro da profundidade L .
- **Corte** (*cutoff*): o limite foi atingido sem encontrar o objetivo; *pode haver* uma solução além do limite L .

A distinção entre *failure* e *cutoff* é crucial: *failure* confirma que não há solução dentro do limite; *cutoff* apenas indica que o limite foi insuficiente.

O Algoritmo 1 apresenta uma implementação recursiva da DLS.

Algorithm 1 DLS-Recursiva(*nó*, *problem*, L)

```
1: if problem.GoalTest(nó.state) then
2:   return solução associada a nó
3: else if nó.profundidade =  $L$  then
4:   return cutoff
5: else
6:   cutoff_ocorreu  $\leftarrow$  False
7:   for all  $a \in \textit{problem.ACTIONS}(\textit{nó.state})$  do
8:     filho  $\leftarrow$  EXPAND(nó,  $a$ )
9:     resultado  $\leftarrow$  DLS-Recursiva(filho, problem,  $L$ )
10:    if resultado = cutoff then
11:      cutoff_ocorreu  $\leftarrow$  True
12:    else if resultado  $\neq$  failure then
13:      return resultado
14:    end if
15:  end for
16:  if cutoff_ocorreu then
17:    return cutoff
18:  else
19:    return failure
20:  end if
21: end if
```

5.2 Exemplo de Execução

Usaremos novamente o grafo da Figura 1, com estado inicial S e estado objetivo G . Exploraremos três valores do limite L para ilustrar os três resultados possíveis da busca em profundidade limitada.

Caso 1 *cutoff* ($L = 1$)

Com $L = 1$, a DLS expande S e gera A e B , ambos na profundidade 1. Ao tentar expandir A , constata que $\text{profundidade}(A) = L$: trata A como folha e retorna *cutoff*. O mesmo ocorre com B . Como pelo menos um *cutoff* foi sinalizado, a DLS retorna *cutoff* ao chamador, indicando que *pode haver* solução além do limite, mas ela não foi encontrada.

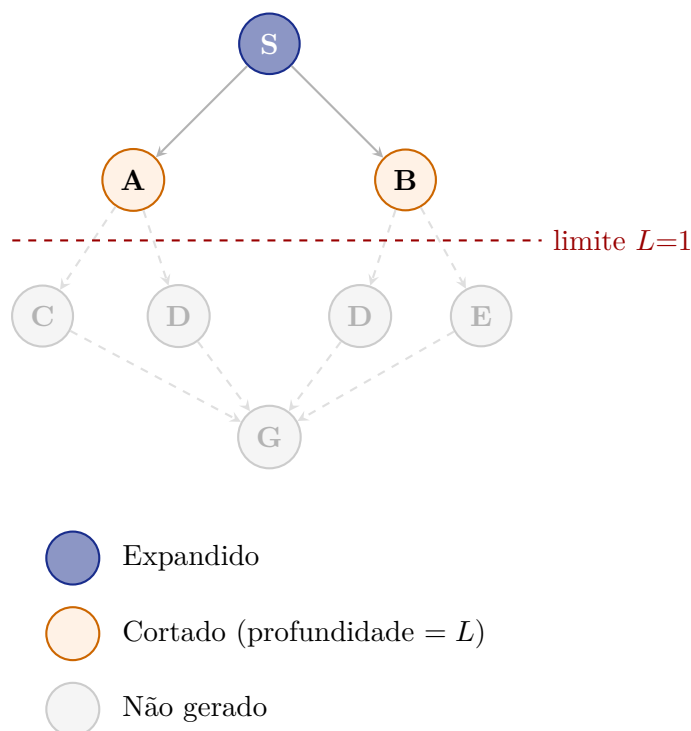


Figura 6: DLS com $L = 1$: resultado *cutoff*. A linha tracejada vermelha marca o limite de profundidade. A e B atingem o limite e são tratados como folhas; seus filhos nunca são gerados. Como G está na profundidade 3, o limite é insuficiente, mas a DLS *não pode afirmar* que não existe solução.

Caso 2 solução encontrada ($L = 3$)

Com $L = 3$, a DLS tem profundidade suficiente para alcançar G . A Tabela ?? mostra a execução, que segue a mesma ordem de expansão da DFS.

Iter.	Nó (prof.)	Ação	Resultado
1	S (0)	Expande; gera A , B	—
2	A (1)	Expande; gera C , D	—
3	C (2)	Expande; gera G	—
4	G (3)	GoalTest: Sim. O teste de corte viria depois.	Solução: $S \rightarrow A \rightarrow C \rightarrow G$

Note que, no Algoritmo 1, o *GoalTest* é aplicado antes do teste de corte por profundidade. Essa ordem é intencional. Se o teste $\text{prof.} = L$ fosse realizado antes do *GoalTest*, um nó objetivo localizado exatamente na profundidade limite poderia ser tratado como *cutoff* e a solução seria perdida.

Por exemplo, neste grafo G está na profundidade 3. Com $L = 3$, quando G é alcançado o algoritmo primeiro verifica *GoalTest*(G), reconhece o objetivo e retorna a solução. Somente se o nó *não* fosse objetivo é que o teste $\text{prof.} = L$ produziria um *cutoff*. Essa ordem de verificação garante que soluções exatamente no limite sejam corretamente identificadas.

Caso 3 *failure* ($L = 3$, objetivo inexistente)

Se o objetivo fosse um estado inexistente no grafo (ou acessível apenas por caminhos mais longos que L em um grafo sem ciclos e sem qualquer ramo cortado no limite), a DLS exploraria todos os ramos até a profundidade 3 sem encontrá-lo e retornaria *failure*, confirmando que *não existe* solução dentro do limite em um espaço acíclico.

É importante distinguir esse caso do seguinte: se existisse algum ramo que atingisse a profundidade L e pudesse continuar além dela, o resultado correto seria *cutoff*, não *failure*. Em outras palavras, *failure* só é apropriado quando a busca pode concluir que não há solução dentro do limite e que nenhum corte por profundidade deixou possibilidades pendentes.

5.3 Propriedades

Completude

A DLS não é completa em geral: se a solução existir a uma profundidade maior que L , ela não será encontrada. Porém, se $L \geq d$, a DLS é completa para a instância.

Otimalidade

A DLS não é ótima: mesmo que encontre uma solução, ela pode não ser a de menor profundidade nem a de menor custo, pois a DLS segue a ordem de expansão da DFS, retornando a primeira solução encontrada dentro do limite.

Complexidade

A complexidade de tempo é $O(b^L)$ e a de espaço é $O(b \cdot L)$, análogas às da DFS com m substituído por L .

O problema da escolha de L

A DLS exige que o projetista forneça o limite L como parâmetro. Mas como escolher um bom valor? Se $L < d$, a DLS retorna *cutoff* mesmo havendo solução. Se $L = d$, é completa para a instância. Se $L > d$, desperdiça esforço explorando níveis desnecessários. Em geral, d não é conhecido de antemão. Quando se conhece o diâmetro do grafo de estados (maior distância entre dois estados), esse valor pode ser usado como limite superior seguro para L .

Pergunta 6

No grafo da Figura 1 com limite $L = 2$, a DLS retorna *cutoff* ao buscar G . Explique a diferença entre os resultados *failure* e *cutoff*, e por que é importante distingui-los em uma implementação prática.

6 Busca em Profundidade Iterativa (IDS)

6.1 Ideia e Implementação

A DLS resolve o problema dos caminhos infinitos, mas exige que se conheça (ou se adivinhe) o limite L adequado. A **Busca em Profundidade Iterativa** (*Iterative Deepening Search*, IDS) elimina essa necessidade aplicando a DLS repetidamente com limites $L = 0, 1, 2, 3, \dots$, aumentando o limite a cada tentativa.

A IDS é conceitualmente simples: para cada $L = 0, 1, 2, \dots$, executa a DLS com limite L . Se a DLS retorna uma solução, a IDS retorna essa solução. Se retorna *failure*, a IDS também retorna *failure*. Se retorna *cutoff*, a IDS incrementa L e tenta novamente. O Algoritmo 2 formaliza esse procedimento.

Algorithm 2 Iterative-Deepening-Search(*problem*)

```
1: for  $L = 0, 1, 2, \dots$  do
2:   resultado  $\leftarrow$  DLS-Recursiva(nó raiz, problem,  $L$ )
3:   if resultado  $\neq$  cutoff then
4:     return resultado
5:   end if
6: end for
```

6.2 Exemplo de Execução

Usaremos novamente o grafo da Figura 1, com estado inicial S e objetivo G (profundidade 3). A Tabela 4 resume o que acontece em cada iteração da IDS.

Iteração (L)	Nós visitados pela DLS	Resultado da DLS
$L = 0$	S	<i>cutoff</i> (S no limite, filhos não gerados)
$L = 1$	S, A, B	<i>cutoff</i> (A e B no limite)
$L = 2$	S, A, C, D, B, D, E	<i>cutoff</i> (C, D, E no limite)
$L = 3$	$S, A, C, \mathbf{G}$	Solução: $S \rightarrow A \rightarrow C \rightarrow G$

Tabela 4: Iterações da IDS no grafo da Figura 1. Nas três primeiras iterações, a DLS retorna *cutoff* e o limite é incrementado. Em $L = 3$, G é encontrado e a busca encerra.

A Figura 7 mostra a árvore de busca construída pela DLS em cada iteração. Observe que, a cada nova iteração, os nós das camadas superiores são reexplorados, mas uma camada adicional é desbloqueada. Na iteração $L = 3$, G finalmente fica dentro do limite e é encontrado.

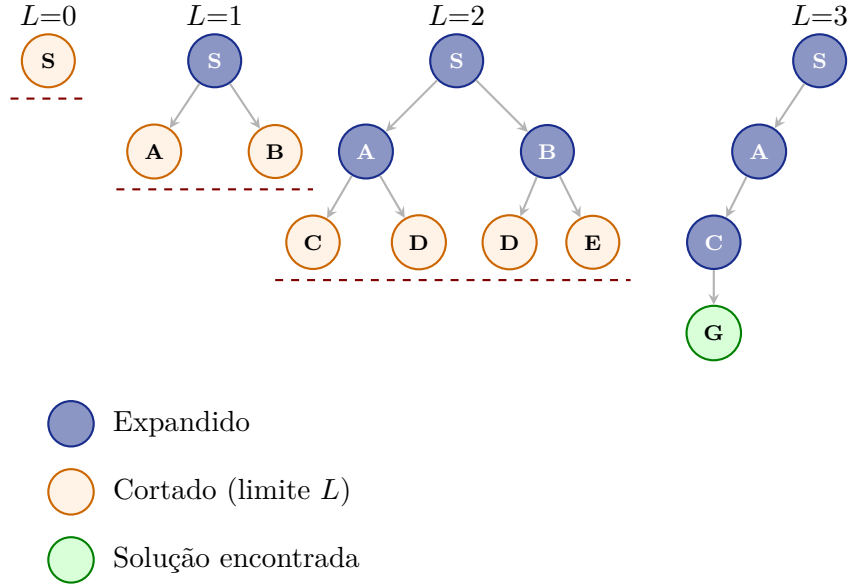


Figura 7: As quatro iterações da IDS no grafo da Figura 1. Nós laranja atingiram o limite e foram tratados como folhas (*cutoff*). Em $L=3$, o nó G (verde) é alcançado e reconhecido como objetivo. O painel $L=3$ mostra apenas o ramo percorrido até a solução; os demais ramos seriam explorados caso G não fosse encontrado primeiro.

6.3 A Questão do Retrabalho

À primeira vista, a IDS parece ineficiente: os nós da profundidade $d - 1$ são gerados duas vezes, os da profundidade $d - 2$ três vezes, e assim por diante. Contudo, esse retrabalho é surpreendentemente pequeno.

O total de nós gerados para encontrar uma solução na profundidade d é:

$$(d + 1)b^0 + db^1 + (d - 1)b^2 + \dots + 1 \cdot b^d = O(b^d).$$

A intuição é que a maior parte dos nós está nas camadas mais profundas. Os nós das camadas superiores, que são repetidos, contribuem pouco para o total. Formalmente, o número de nós da IDS é no máximo $b/(b - 1)$ vezes o da BFS na mesma profundidade, uma constante multiplicativa irrelevante para a análise assintótica.

Para tornar isso concreto, considere $b = 2$ e $d = 3$. A fórmula acima dá:

$$4 \cdot 1 + 3 \cdot 2 + 2 \cdot 4 + 1 \cdot 8 = 4 + 6 + 8 + 8 = 26 \text{ nós.}$$

A BFS geraria $1 + 2 + 4 + 8 = 15$ nós até a mesma profundidade. A IDS gera apenas $26/15 \approx 1,7\times$ mais nós, uma sobrecarga modesta, especialmente considerando que a IDS usa espaço $O(b \cdot d)$ enquanto a BFS exige $O(b^d)$.

6.4 Propriedades

Compleitude

A IDS é **completa** se o fator de ramificação b for finito. Como ela executa uma DLS com limite crescente, todo estado alcançável a qualquer profundidade finita será eventualmente visitado.

Otimidade

A IDS é **ótima quando o custo de cada ação é igual a 1**: como os limites crescem de 1 em 1, a primeira solução encontrada está necessariamente no nível mais raso. Para custos variáveis, a IDS pode retornar uma solução subótima, pois um caminho mais profundo e mais barato poderia existir. Nesses casos, a estratégia adequada é a UCS.

Complexidade

A **complexidade de tempo** é $O(b^d)$, a mesma que a BFS assintoticamente. A **complexidade de espaço** é $O(b \cdot d)$, a mesma que a DFS: em qualquer ponto da execução, apenas o caminho atual e os irmãos não visitados ao longo dele precisam estar em memória.

Por que a IDS combina BFS e DFS

A IDS herda da BFS a completude e a otimalidade para custos uniformes: como os limites crescem de 1 em 1, a primeira solução encontrada está na menor profundidade possível. Ao mesmo tempo, herda da DFS o uso linear de memória, pois cada execução individual da IDS mantém apenas o caminho atual e seus irmãos pendentes.

IDS: a melhor estratégia de busca cega

Para a maioria dos problemas práticos com espaço de estados grande, custo uniforme e d desconhecido, a IDS é a **estratégia de busca sem informação preferida**. Ela é completa, ótima (custo unitário), usa espaço linear e tem o mesmo custo assintótico de tempo que a BFS, sem o catastrófico requisito de memória desta última.

Pergunta 7

A IDS reexpande nós gerados em iterações anteriores. (i) Calcule o número total de nós gerados pela IDS para encontrar uma solução na profundidade $d = 4$ com fator de ramificação $b = 2$, usando a fórmula $(d+1)b^0 + db^1 + \dots + b^d$. (ii) Compare com o número de nós gerados pela BFS na mesma profundidade. (iii) O retrabalho da IDS é aceitável?

Pergunta 8

Explique por que a IDS **não** é ótima quando as ações têm custos diferentes. Dê um exemplo simples em que a IDS retorna uma solução mais cara do que a solução ótima. Qual algoritmo deveria ser usado no lugar da IDS nesse caso?

7 Comparação entre Estratégias

A Tabela 5 sumariza as propriedades dos cinco algoritmos estudados. As seções seguintes comentam as linhas mais importantes da tabela e oferecem um guia prático de escolha.

Critério	BFS	UCS	DFS	DLS	IDS
Completa?	Sim ^a	Sim ^b	Não	Não	Sim ^a
Ótima?	Sim ^c	Sim	Não	Não	Sim ^c
Tempo	$O(b^d)$	$O(b^{C^*/\varepsilon})$	$O(b^m)$	$O(b^L)$	$O(b^d)$
Espaço	$O(b^d)$	$O(b^{C^*/\varepsilon})$	$O(bm)$	$O(bL)$	$O(bd)$

^a Se b for finito. ^b Se todos os custos $\geq \varepsilon > 0$. ^c Se todos os custos forem iguais.

Tabela 5: Comparação das estratégias de busca sem informação. b : fator de ramificação; d : profundidade da solução mais rasa; m : profundidade máxima; L : limite (DLS); C^* : custo da solução ótima; ε : custo mínimo de uma ação.

Leitura da tabela

A linha de completude revela que apenas BFS, UCS e IDS garantem encontrar uma solução quando ela existe. DFS e DLS podem falhar mesmo em problemas com solução: a DFS por entrar em ciclos ou ramos infinitos, a DLS por impor um limite insuficiente.

Ao comparar essas propriedades, vale lembrar que os *exemplos de execução* da nota não usam exatamente a mesma variante operacional para todos os algoritmos: BFS e UCS foram ilustradas como busca em grafo, enquanto DFS, DLS e IDS foram ilustradas como busca em árvore com *path-check*. Essa escolha afeta a execução passo a passo dos exemplos, mas não altera as propriedades assintóticas resumidas na tabela.

A linha de otimalidade mostra que UCS é o único algoritmo ótimo em geral, válido para qualquer custo não-negativo. BFS e IDS também são ótimas, mas apenas quando todos os custos são iguais, pois nesse caso profundidade mínima coincide com custo mínimo.

As linhas de complexidade revelam o compromisso fundamental desta área. Tempo e espaço caminham juntos para BFS e UCS: ambos crescem exponencialmente, tornando esses algoritmos inviáveis para problemas com d ou C^*/ε grandes. A DFS quebra esse padrão ao usar espaço linear $O(bm)$, mas paga o preço de não ser completa nem ótima. A IDS é a única estratégia que combina espaço linear com completude e otimalidade: ela usa $O(bd)$ de espaço, o mesmo que a DFS, mas tem o mesmo desempenho assintótico de tempo que a BFS.

Guia prático de escolha

A Tabela 6 traduz as propriedades em critérios de escolha para os cenários mais comuns.

Vale destacar duas comparações que a tabela não torna imediatamente visíveis. Primeiro, BFS e IDS têm o mesmo desempenho assintótico de tempo, mas a IDS usa exponencialmente menos memória: para $b = 10$ e $d = 8$, a BFS exige cerca de 10^8 nós em memória simultaneamente, enquanto a IDS mantém apenas $10 \times 8 = 80$ nós. Segundo, UCS pode ser significativamente mais lenta que BFS quando existem muitas ações de custo baixo, pois $\lfloor C^*/\varepsilon \rfloor$ pode ser muito maior que d .

Cenário	Algoritmo	Motivo
Custos variáveis; solução ótima obrigatória	UCS	Otimidade com custos heterogêneos
Custos uniformes; memória limitada; d desconhecido	IDS	Completude, otimalidade e espaço linear
Custos uniformes; memória abundante; necessidade de implementação simples	BFS	Implementação direta e comportamento em camadas
Qualquer solução aceitável; memória muito restrita; espaço acíclico	DFS	Espaço linear e implementação recursiva simples
d aproximadamente conhecido; qualquer solução aceitável	DLS	Limite explícito de profundidade e custo controlado

Tabela 6: Guia prático de escolha entre as estratégias de busca sem informação.

Pergunta 9

Com base na Tabela 5, responda: (i) Em qual cenário você escolheria BFS em vez de IDS? (ii) Em qual cenário DFS seria a melhor opção, apesar de não ser completa nem ótima? (iii) Qual algoritmo você escolheria para um problema com custos variáveis e sem limite de memória? Justifique cada resposta em uma ou duas frases.

8 Considerações Finais

As estratégias de busca sem informação representam a base sobre a qual toda a teoria de busca em IA é construída. Elas não exigem nenhum conhecimento adicional além da formulação do problema, e ainda assim permitem resolver instâncias de tamanho moderado de forma sistemática e garantida.

A principal lição desta unidade não é decorar as propriedades de cada algoritmo, mas entender *por que* essas propriedades decorrem das escolhas de projeto. Cada propriedade (completude, otimalidade, complexidade de tempo, complexidade de espaço) é consequência direta de uma decisão sobre como gerenciar a fronteira. Fila FIFO implica expansão em largura, o que dá completude e otimalidade com custo uniforme, mas memória exponencial. Pilha LIFO implica expansão em profundidade, com memória linear, mas sem garantias de completude ou otimalidade. Fila de prioridade por custo acumulado implica expansão em ordem de custo crescente, garantindo otimalidade geral. A combinação DLS com limite crescente produz a IDS, que reúne as melhores propriedades de BFS e DFS.

As estratégias cegas têm um limite fundamental: a complexidade exponencial de tempo inviabiliza a solução de instâncias grandes. A alternativa é usar *estratégias de busca com informação* (busca heurística), que incorporam estimativas do custo restante até o objetivo para guiar a exploração de forma mais inteligente. O algoritmo A^* é o exemplo mais importante dessa classe e será tema das próximas aulas.

Pergunta 10

Considere um problema de busca com $b = 10$ e $d = 10$. (i) Quantos nós seriam expandidos no pior caso pela BFS? (ii) Quantos pela IDS? (iii) Suponha que existisse um oráculo que nos dissesse, para cada estado, uma estimativa perfeita da distância até o objetivo. Como isso poderia reduzir o número de nós expandidos? Esse é o ponto de partida para a busca heurística, tema das próximas aulas.

9 Atividades Intra-Classe

Nos exercícios a seguir, utilize o grafo da Figura 1, já familiar das seções anteriores, com estado inicial S e estado objetivo G .

9.1 Exercício 1: BFS Manual

Execute manualmente a BFS a partir de S . Para cada iteração, indique: (a) o nó selecionado para expansão; (b) o conteúdo da fronteira (fila FIFO) após a expansão. Quantos nós foram expandidos antes de encontrar G ? O resultado coincide com o da Tabela 1?

9.2 Exercício 2: DFS Manual

Execute manualmente a DFS a partir de S . Para cada iteração, indique: (a) o nó selecionado para expansão; (b) o conteúdo da pilha LIFO após a expansão. A solução encontrada é a mesma que a da BFS? Justifique por que sim ou por que não.

9.3 Exercício 3: UCS Manual

Considere o grafo da Figura 1 com os seguintes custos de aresta: $S-A$: 1, $S-B$: 5, $A-C$: 3, $A-D$: 2, $B-D$: 1, $B-E$: 4, $C-G$: 6, $D-G$: 2, $E-G$: 3. Execute a UCS registrando a fila de prioridade a cada iteração. Qual caminho é encontrado e qual é seu custo total? Esse caminho coincide com o encontrado pela BFS no Exercício 1?

9.4 Exercício 4: DLS e IDS Manuais

Considere o grafo da Figura 1 com custos uniformes.

- (a) Execute a DLS com limite $L = 1$. Qual é o resultado retornado: solução, *failure* ou *cutoff*? Justifique.
- (b) Execute a DLS com limite $L = 2$. Qual é o resultado?
- (c) Execute a IDS até encontrar G . Para cada iteração, indique o valor de L e o resultado da DLS correspondente. Em qual iteração G é encontrado?

9.5 Exercício 5: Escolha do Algoritmo

Para cada cenário abaixo, indique qual estratégia de busca sem informação você usaria e justifique em duas frases.

- (a) O espaço de estados é muito grande, as ações têm custo uniforme, e a profundidade da solução é desconhecida.
- (b) O espaço de estados é pequeno, as ações têm custos variados, e é essencial encontrar a solução de menor custo.
- (c) A memória disponível é muito limitada, e qualquer solução (não necessariamente ótima) é aceitável.
- (d) O espaço é acíclico, custo uniforme, e é fundamental encontrar a solução de menor profundidade com mínimo uso de memória.

Pergunta 11

Considere a IDS aplicada a um problema em que a solução está na profundidade $d = 5$ com $b = 3$. (i) Quantas vezes o nó raiz é expandido? (ii) Quantas vezes os nós da profundidade 1 são expandidos? (iii) Calcule o número total de nós gerados pela IDS até encontrar a solução e compare com b^d . Essa comparação justifica a afirmação de que o retrabalho da IDS é aceitável?

Pergunta 12

Algoritmos de busca ainda ocupam uma posição fundamental no projeto de agentes inteligentes. No entanto, todas as estratégias cegas estudadas têm complexidade exponencial de tempo. Discuta: em que tipos de problema prático as estratégias cegas são viáveis, e em que tipos elas se tornam impraticáveis? O que, em sua opinião, seria necessário para tornar a busca viável em problemas com espaços de estados da ordem de 10^{20} estados?